

RFID

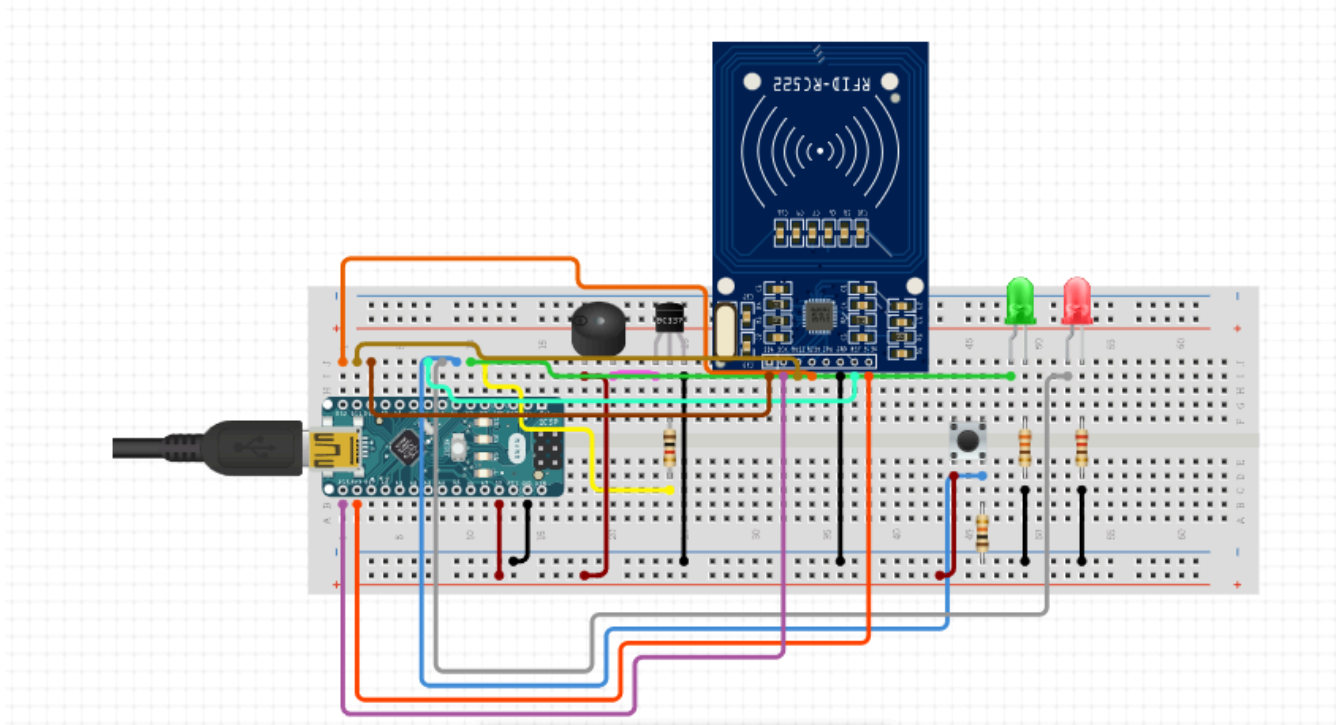
Name : Deepraj Naik

Roll No : 2519

● Components Required

Sr.No.	Component	Quantity	Description	Use
1.	Arduino Nano	1	It is a small microcontroller board.	It is used to control the whole RFID attendance system.
2.	BreadBoard	1	It is a board used to connect components without soldering.	It is used to make the circuit connections.
3.	LED	2	It is a small light that glows when electricity passes through it.	One LED is used for verified students and the other for unverified students.
4.	Buzzer	1	It is a small device that produces a sound.	It is used to give a sound indication when a card is scanned.
5.	Pushbutton Switch	1	It is a button used to give input to the Arduino.	It is used to start and stop storing RFID cards for attendance.
6.	Resistor	3	It is a component that controls the flow of current.	It is used to protect the LEDs and other components from high current.
7.	Rfid reader	1	It is a device that reads information from an RFID card or tag.	It is used to scan the RFID card and check the student.
8.	Rfid card/tag	1	It is a card or tag that contains a unique ID.	It is used by students to scan and mark their attendance.

- **Circuit Diagram**



- **Program Code**

```
#include <SPI.h>
#include <MFRC522.h>
#include <EEPROM.h>

// PINS

#define SS_PIN    10
#define RST_PIN   9

#define BUZZER_PIN 5
#define GREEN_LED  6
#define RED_LED    7
#define BUTTON_PIN 2

// RFID

MFRC522 rfid(SS_PIN, RST_PIN);

// EEPROM

#define MAX_CARDS 20
#define UID_SIZE 10
```

```
int cardCount = 0;

// MODE

bool registrationMode = false;

// ATTENDANCE

bool attendanceMarked[MAX_CARDS];

//
// SETUP
//

void setup() {

    Serial.begin(9600);

    // RFID
    SPI.begin();
    rfid.PCD_Init();

    // Outputs
    pinMode(BUZZER_PIN, OUTPUT);
    pinMode(GREEN_LED, OUTPUT);
    pinMode(RED_LED, OUTPUT);

    // Button
    pinMode(BUTTON_PIN, INPUT_PULLUP);

    digitalWrite(GREEN_LED, LOW);
    digitalWrite(RED_LED, LOW);
    noTone(BUZZER_PIN);

    // Read registered cards
    cardCount = EEPROM.read(0);

    if (cardCount > MAX_CARDS) {
        cardCount = 0;
        EEPROM.update(0, 0);
    }

    // Clear attendance
```

```

for (int i = 0; i < MAX_CARDS; i++) {
    attendanceMarked[i] = false;
}

Serial.println();
Serial.println("=====");
Serial.println("    RFID ATTENDANCE SYSTEM");
Serial.println("=====");

Serial.print("Registered Students: ");
Serial.println(cardCount);

Serial.println();
Serial.println("ATTENDANCE MODE");
Serial.println("Press BUTTON to enter registration mode.");
Serial.println();

delay(1000);
}

//
// MAIN LOOP
//

void loop() {

    // Check button FIRST
    checkButton();

    // Check RFID
    if (!rfid.PICC_IsNewCardPresent()) {
        return;
    }

    if (!rfid.PICC_ReadCardSerial()) {
        return;
    }

    Serial.println("-----");

    Serial.print("Card UID: ");
    printUID();
    Serial.println();
}

```

```

// Registration
if (registrationMode) {
    registerCard();
}

// Attendance
else {
    checkAttendance();
}

rfid.PICC_HaltA();
rfid.PCD_StopCrypto1();

delay(500);
}

//
// BUTTON
//

void checkButton() {

    // Button uses INPUT_PULLUP
    // HIGH = not pressed
    // LOW = pressed

    if (digitalRead(BUTTON_PIN) == LOW) {

        // Wait for button to settle
        delay(50);

        // Make sure button is still pressed
        if (digitalRead(BUTTON_PIN) == LOW) {

            // Toggle mode
            registrationMode = !registrationMode;

            Serial.println();
            Serial.println("=====");

            if (registrationMode == true) {

```

```

Serial.println("REGISTRATION MODE: ON");
Serial.println();
Serial.println("Scan RFID card to REGISTER.");
Serial.println("Press button again to STOP registration.");
Serial.println();

// Green LED + beep
digitalWrite(GREEN_LED, HIGH);
tone(BUZZER_PIN, 1200);
delay(200);
noTone(BUZZER_PIN);
digitalWrite(GREEN_LED, LOW);

} else {

Serial.println("REGISTRATION MODE: OFF");
Serial.println();
Serial.println("ATTENDANCE MODE: ON");
Serial.println("Only registered students are accepted.");
Serial.println();

// Red LED + beep to indicate mode change
digitalWrite(RED_LED, HIGH);
tone(BUZZER_PIN, 800);
delay(200);
noTone(BUZZER_PIN);
digitalWrite(RED_LED, LOW);
}

Serial.println("=====");
Serial.println();

// IMPORTANT:
// Wait until button is released
while (digitalRead(BUTTON_PIN) == LOW) {
    delay(10);
}

// Small delay after release
delay(100);
}
}
}

```

```

//
// PRINT UID
//

void printUID() {

    for (byte i = 0; i < rfid.uid.size; i++) {

        if (rfid.uid.uidByte[i] < 0x10) {
            Serial.print("0");
        }

        Serial.print(rfid.uid.uidByte[i], HEX);

        if (i < rfid.uid.size - 1) {
            Serial.print(" ");
        }
    }
}

//
// REGISTER CARD
//

void registerCard() {

    int existingCard = findCard();

    // Already registered
    if (existingCard != -1) {

        Serial.println("STATUS: ALREADY REGISTERED");

        Serial.print("Student Number: ");
        Serial.println(existingCard + 1);

        Serial.println("This card is already stored.");

        // Two beeps
        tone(BUZZER_PIN, 1200);
        delay(150);
        noTone(BUZZER_PIN);
    }
}

```

```

    delay(100);

    tone(BUZZER_PIN, 1200);
    delay(150);
    noTone(BUZZER_PIN);

    greenFlash();

    return;
}

// Memory full
if (cardCount >= MAX_CARDS) {

    Serial.println("ERROR: REGISTRATION FULL");

    errorBeep();

    return;
}

// EEPROM address
int address = 1 + (cardCount * UID_SIZE);

// Save UID
for (byte i = 0; i < UID_SIZE; i++) {

    if (i < rfid.uid.size) {
        EEPROM.update(address + i, rfid.uid.uidByte[i]);
    }
    else {
        EEPROM.update(address + i, 0);
    }
}

// Increase count
cardCount++;

EEPROM.update(0, cardCount);

Serial.println("STATUS: REGISTRATION SUCCESS");

Serial.print("Student Number: ");

```

```

Serial.println(cardCount);

Serial.println("RFID card stored successfully.");

Serial.print("Total Registered: ");
Serial.println(cardCount);

// Success
tone(BUZZER_PIN, 1500);
delay(200);
noTone(BUZZER_PIN);

greenFlash();

Serial.println();
Serial.println("Scan another student card.");
}

//
// ATTENDANCE
//

void checkAttendance() {

    int studentNumber = findCard();

    //
    // VERIFIED
    //

    if (studentNumber != -1) {

        Serial.println("STATUS: VERIFIED");

        Serial.print("Student Number: ");
        Serial.println(studentNumber + 1);

        // Already marked
        if (attendanceMarked[studentNumber]) {

            Serial.println("ATTENDANCE: ALREADY MARKED");

            digitalWrite(RED_LED, HIGH);

```

```

    tone(BUZZER_PIN, 700);
    delay(200);
    noTone(BUZZER_PIN);

    delay(500);

    digitalWrite(REDA_LED, LOW);

    return;
}

// Mark attendance
attendanceMarked[studentNumber] = true;

Serial.println("ATTENDANCE: MARKED");
Serial.println("Student is PRESENT.");

// Green LED
digitalWrite(GREEN_LED, HIGH);
digitalWrite(REDA_LED, LOW);

// Success beep
tone(BUZZER_PIN, 1500);
delay(200);
noTone(BUZZER_PIN);

delay(1000);

digitalWrite(GREEN_LED, LOW);
}

//
// NOT VERIFIED
//

else {

    Serial.println("STATUS: NOT VERIFIED");

    Serial.println("ATTENDANCE: DENIED");

    Serial.println("Student is NOT registered.");

```

```

digitalWrite(GREEN_LED, LOW);
digitalWrite(RED_LED, HIGH);

// Three warning beeps
errorBeep();

delay(1000);

digitalWrite(RED_LED, LOW);
}
}

//
// FIND CARD
//

int findCard() {

  for (int card = 0; card < cardCount; card++) {

    int address = 1 + (card * UID_SIZE);

    bool match = true;

    for (byte i = 0; i < UID_SIZE; i++) {

      byte storedByte = EEPROM.read(address + i);

      byte scannedByte = 0;

      if (i < rfid.uid.size) {
        scannedByte = rfid.uid.uidByte[i];
      }

      if (storedByte != scannedByte) {

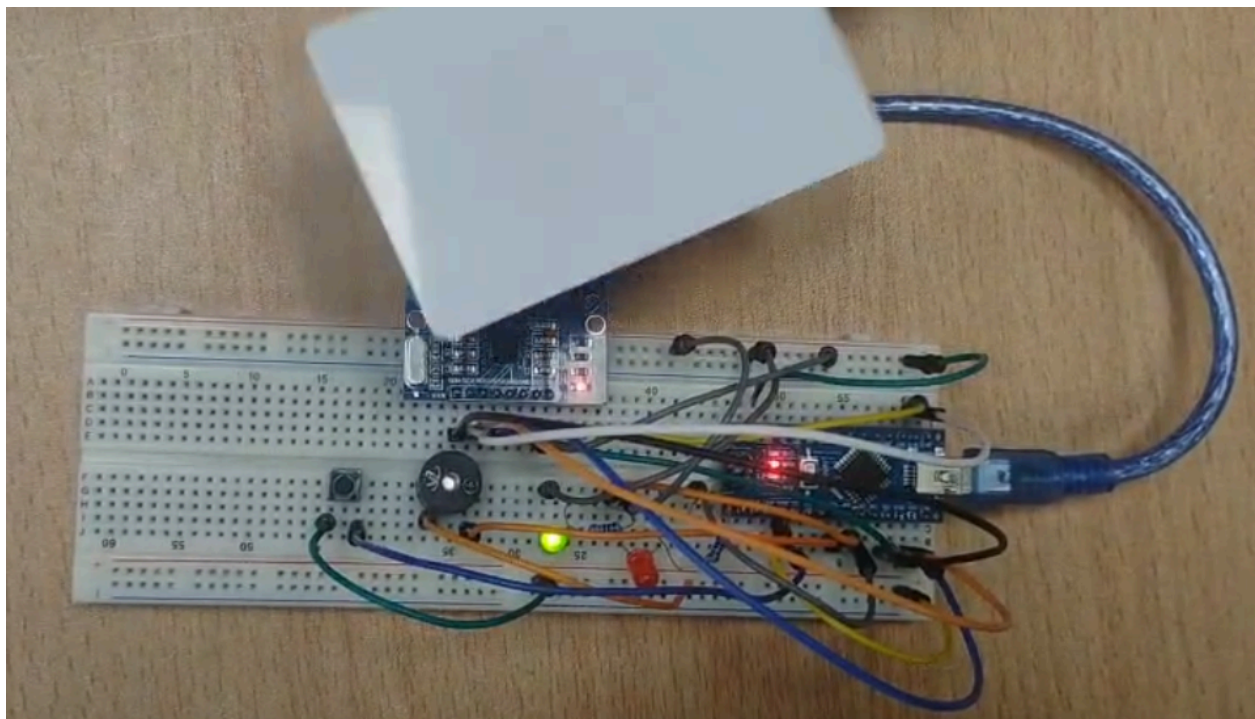
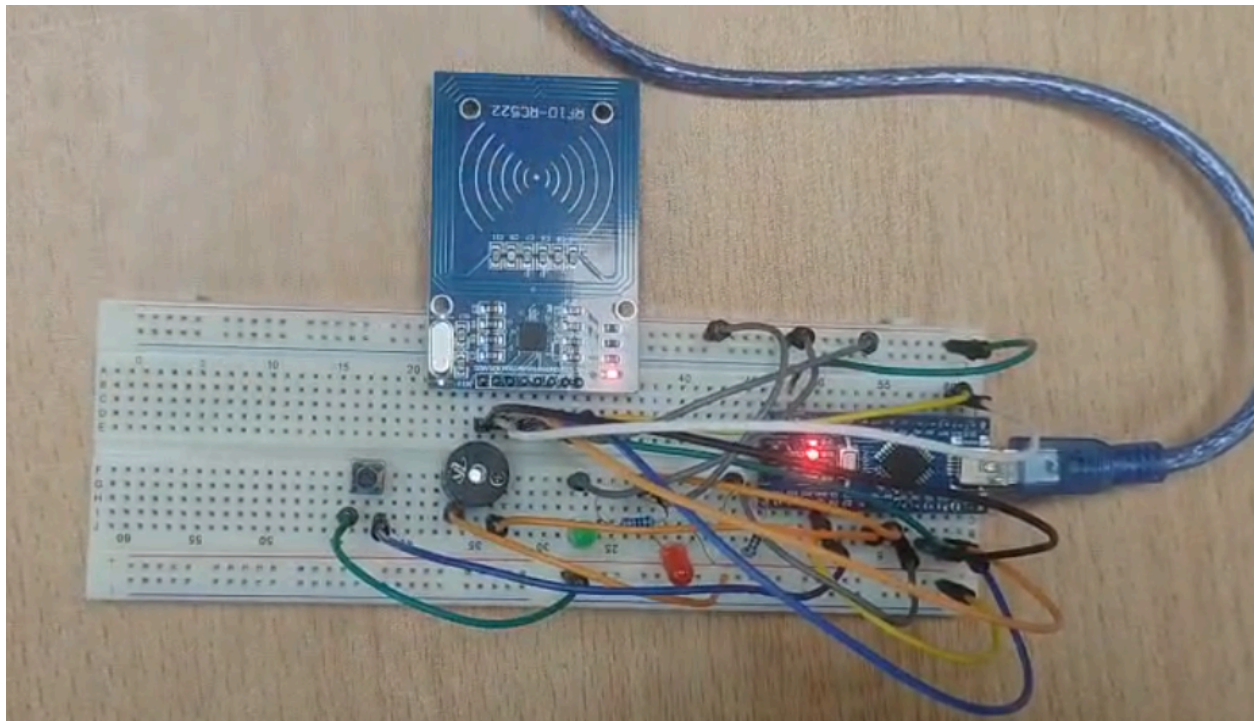
        match = false;
        break;
      }
    }

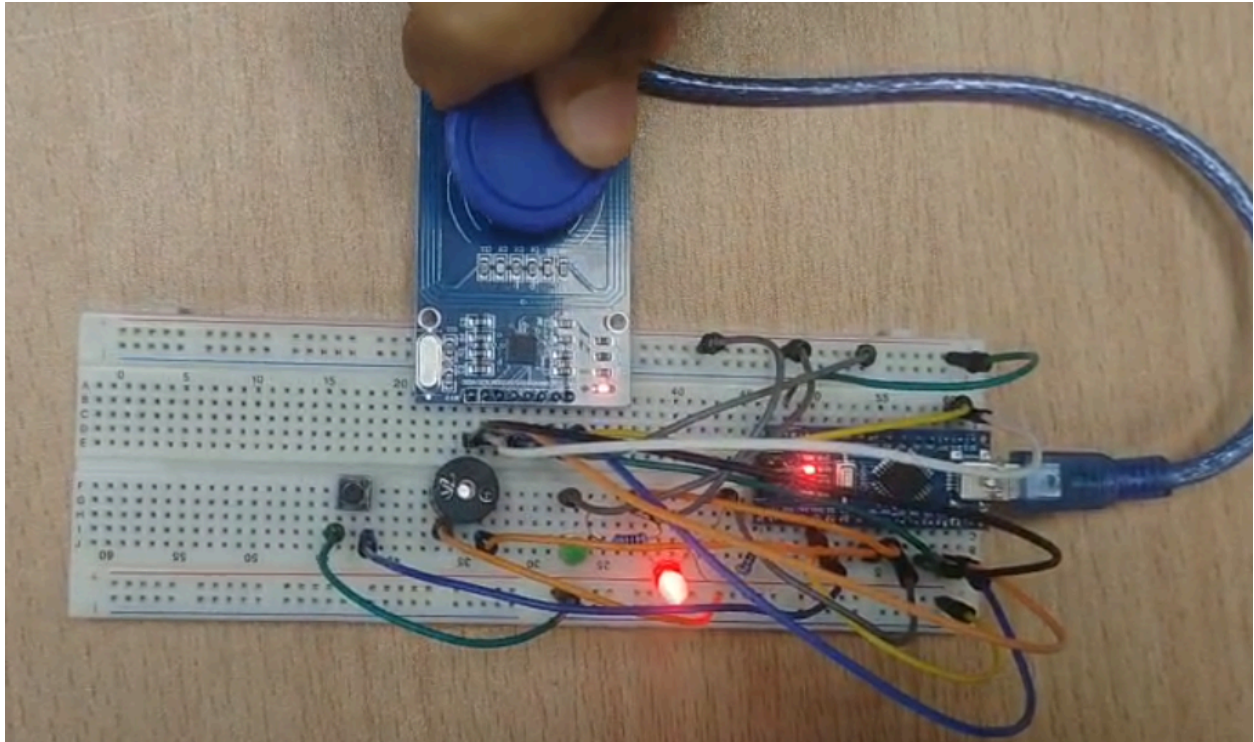
    if (match) {
      return card;
    }
  }
}

```

```
    }  
  }  
  
  return -1;  
}  
  
//  
// GREEN LED  
//  
  
void greenFlash() {  
  
  digitalWrite(GREEN_LED, HIGH);  
  delay(400);  
  digitalWrite(GREEN_LED, LOW);  
}  
  
//  
// ERROR BEEP  
//  
  
void errorBeep() {  
  
  for (int i = 0; i < 3; i++) {  
  
    tone(BUZZER_PIN, 600);  
    delay(150);  
  
    noTone(BUZZER_PIN);  
    delay(100);  
  }  
}
```

- Photograph of the Experiment





- **Conclusion**

In this experiment, we saw how to make an RFID attendance system using Arduino Nano. We learned how to read the unique ID of an RFID card using an RFID reader. We also learned how to use a pushbutton to start and stop storing the student cards. The system allows verified students to mark their attendance and denies unverified students. We also learned how to use LEDs and a buzzer to show whether the student is verified or not.